

JavaScriptの `map` と `filter` をわかりやすく解説

配列を扱う際に最も使用頻度の高いメソッドが `map` と `filter` です。役割を混同しやすいので、明確に整理します。

1. まず結論（一言でいうと）

メソッド	一言でいうと	返り値の配列の長さ
<code>map</code>	各要素を 変換 する	元の配列と同じ長さ
<code>filter</code>	条件に合う要素だけ 選び出す	条件次第で 短くなる (または変わらない)

👉 「全部の形を変えたい」なら `map`、「一部だけ取り出したい」なら `filter`、というのが一番の判断基準です。

2. アナロジーで理解する 🏭

- `map` = ベルトコンベアの上を通る商品**全部**に、同じ加工(スタンプ押し、包装など)を施す工程
→ 個数は変わらないが、中身(見た目)が変わる
- `filter` = ベルトコンベアの上を通る商品から、**検品担当が不良品だけ弾く**工程
→ 中身は変わらないが、個数が減る

`map`: [🍎, 🍎, 🍎] → (皮をむく) → [🍏, 🍏, 🍏] ← 3個のまま、中身が変化
`filter`: [🍎, 🍏, 🍎] → (赤いのだけ残す) → [🍎, 🍎] ← 個数が変化、中身はそのまま

3. `map` : 配列の各要素を変換する

基本構文

```
const 新しい配列 = 元の配列.map((要素, index) => {  
  return 変換後の値;  
});
```

基本例

```
const numbers = [1, 2, 3, 4];  
  
const doubled = numbers.map((n) => n * 2);  
console.log(doubled); // [2, 4, 6, 8]  
console.log(numbers); // [1, 2, 3, 4] ← 元の配列は変化しない
```

オブジェクトの配列を扱う例(実務でよく使う形)

```
const todos = [  
  { id: 1, text: "牛乳を買う", done: false },  
  { id: 2, text: "掃除する", done: false },  
];  
  
// id=2 のtodoだけ、doneをtrueに変える
```

```
const updated = todos.map((t) =>
  t.id === 2 ? { ...t, done: true } : t
);

console.log(updated);
// [
//   { id: 1, text: "牛乳を買う", done: false },
//   { id: 2, text: "掃除する", done: true }, ← ここだけ変わった
// ]
```

ポイント: 対象外の要素は `t` をそのまま返すことで、「変更しない」を表現しています。`map` は必ず元と同じ数の要素を返すため、「変えない要素」も明示的に返してあげる必要があります。

4. `filter` : 条件に合う要素だけ残す

基本構文

```
const 新しい配列 = 元の配列.filter((要素, index) => {
  return true or false を返す条件;
});
```

コールバック関数が `true` を返した要素だけが新しい配列に残ります。

基本例

```
const numbers = [1, 2, 3, 4, 5, 6];

const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4, 6]
```

オブジェクトの配列を扱う例(実務でよく使う形)

```
const todos = [
  { id: 1, text: "牛乳を買う", done: true },
  { id: 2, text: "掃除する", done: false },
  { id: 3, text: "メールを送る", done: true },
];

// 未完了(done: false)のtodoだけ残す
const notDone = todos.filter((t) => !t.done);
console.log(notDone);
// [{ id: 2, text: "掃除する", done: false }]

// id=1 のtodoを削除したい(=id=1以外を残す)
const removed = todos.filter((t) => t.id !== 1);
console.log(removed);
// [
//   { id: 2, text: "掃除する", done: false },
//   { id: 3, text: "メールを送る", done: true },
// ]
```

ポイント: 「削除する」という操作は、実際には「削除したい要素以外を残した新しい配列を作る」という発想で実現します。`splice` のように元の配列から直接取り除くわけではありません。

5. `map` と `filter` の比較表

	<code>map</code>	<code>filter</code>
コールバックの返回值	変換後の値(何でもよい)	<code>true / false</code> (booleanのみ)
配列の長さ	変わらない	条件次第で減る
主な用途	更新・加工・整形	検索・絞り込み・削除
Reactでの典型例	Todoの中身を編集する	Todoを削除する、完了/未完了で絞り込む

```
// 更新(map): 中身を書き換える
setTodos(todos.map((t) => (t.id === id ? { ...t, text: trimmed } : t)));

// 削除(filter): 対象を取り除く
setTodos(todos.filter((t) => t.id !== id));

// 絞り込み(filter): 表示するものだけ選ぶ
const visibleTodos = todos.filter((t) => !t.done);
```

6. 組み合わせて使うこともできる

`map` と `filter` はチェーン(連結)して使うことも多いです。

```
const todos = [
  { id: 1, text: "牛乳を買う", done: true },
  { id: 2, text: "掃除する", done: false },
  { id: 3, text: "メールを送る", done: false },
];

// ①未完了のものだけ残し、②テキストの先頭に「TODO:」を付ける
const result = todos
  .filter((t) => !t.done)
  .map((t) => ({ ...t, text: `TODO: ${t.text}` }));

console.log(result);
// [
//   { id: 2, text: "TODO: 掃除する", done: false },
//   { id: 3, text: "TODO: メールを送る", done: false },
// ]
```

処理の流れが「まず絞り込んでから、次に加工する」という形で、上から下に読める点が読みやすさのポイントです。

7. ⚠ よくある間違い

間違い①: `filter` のコールバックがboolean以外を返している

```
// 悪い例: 条件のつもりが値を返してしまっている
const result = todos.filter((t) => t.id); // idが0の要素は除外されてしまう(falsy)

// 良い例: 明示的にboolean比較にする
const result = todos.filter((t) => t.id === targetId);
```

間違い②: `map` で「フィルタリング」しようとする

```
// 悪い例: mapの中でnullを返して「除外」しようとしている
const result = todos.map((t) => (t.done ? null : t)); // nullが残ってしまう!
console.log(result); // [null, {...}, null] ← 除外されず、nullが混ざる

// 良い例: 絞り込みたいならfilterを使う
const result = todos.filter((t) => !t.done);
```

`map` は「除外」ができません。あくまで「変換」専用です。要素を減らしたい時は必ず `filter` を使います。

間違い③: `forEach` と `map` を混同する

```
// forEach: 値を返さない(新しい配列を作らない)。ただ処理を実行するだけ
todos.forEach((t) => t.text); // 何も返ってこない(undefined)

// map: 新しい配列を作りたい時に使う
const texts = todos.map((t) => t.text); // ["牛乳を買う", "掃除する", ...]
```

「新しい配列が欲しいのに `forEach` を使ってしまう」というのもよくあるミスです。

8. まとめ

- `map` = 全要素を「変換」する。個数は変わらない
- `filter` = 条件に合う要素だけ「選別」する。個数が変わりうる
- どちらも元の配列を変更せず、新しい配列を返す(Reactのイミュータブルな更新ルールと相性が良い)
- 「更新なら `map`、削除・絞り込みなら `filter`」と覚えると使い分けやすい
- `filter().map()` のようにチェーンして「絞り込み→加工」を1行で表現することもできる